

Tokenization

The Financial Times is

Word embeddings, positional encoding



Introductory Machine Learning

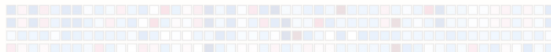
Transformers

Self-Attention

April 8



Encoded output



Reminders

- Assignment 4 due tomorrow at midnight
- Assignment 5 (last!) is out
- Final exam, May 4 at 2pm
- Review sessions:
 - ▶ Yihan: Wed. April 29, 6pm
 - ▶ Dingyao: Thu. April 30, 8pm
 - ▶ Awni: Friday, May 1, 3pm

For today

- Transformers
- Attention
- Scaling laws

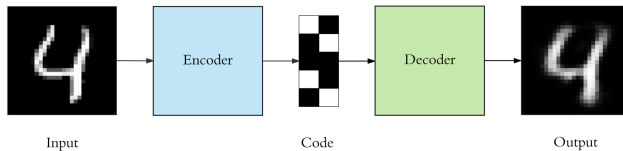
Quick Recap

Autoencoders

Recall: Autoencoders

- Unsupervised learning methods
- Squeeze high dimensional data through a “bottleneck” of lower dimension
- Train to minimize reconstruction error

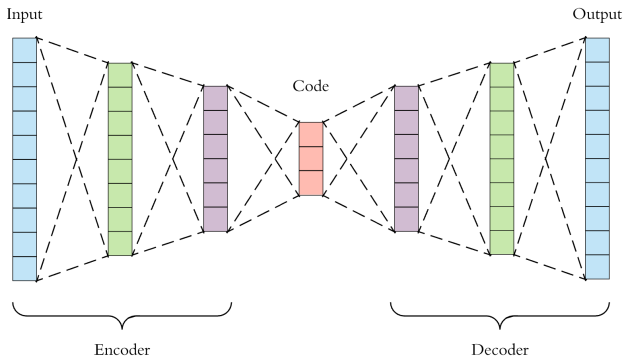
Autoencoders



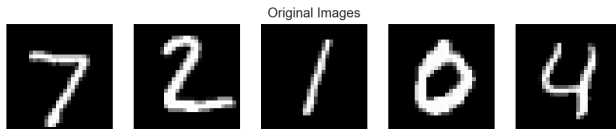
Important aspects

- Unsupervised: No labels used, discovers useful features of input
- Compression: Code reduces dimension of data
- Lossy: Input won't be reconstructed exactly
- Trained: The compression algorithm is learned for specific data
- *Not generative models*

Deep architecture



MNIST data

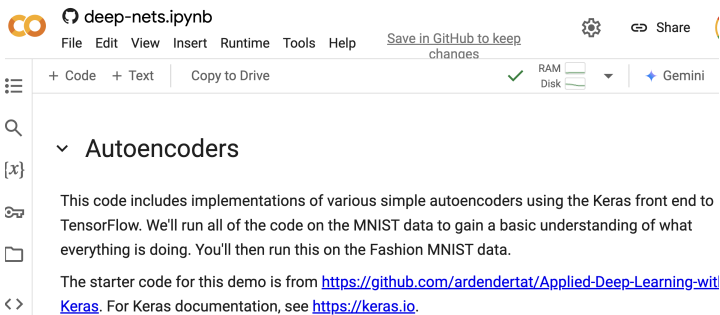


$$28 \times 28 = 784 = D$$

Implementation using Keras

```
1  input_size = 784
2  hidden_size = 128
3  code_size = 32
4  
5  input_img = Input(shape=(input_size,))
6  hidden_1 = Dense(hidden_size, activation='relu')(input_img)
7  code = Dense(code_size, activation='relu')(hidden_1)
8  hidden_2 = Dense(hidden_size, activation='relu')(code)
9  output_img = Dense(input_size, activation='sigmoid')(hidden_2)
10
11 autoencoder = Model(input_img, output_img)
12 autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
13 autoencoder.fit(x_train, x_train, epochs=5)
```

Demo



The screenshot shows a JupyterLab interface. At the top, the notebook title is 'deep-nets.ipynb'. Below the title bar, there are tabs for 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. To the right of these tabs, there is a link that says 'Save in GitHub to keep changes', a gear icon for settings, and a 'Share' button. Below the tabs, there is a toolbar with '+ Code', '+ Text', and 'Copy to Drive'. To the right of the toolbar, there is a green checkmark, a 'RAM' indicator, a 'Disk' indicator, and a 'Gemini' button. On the left side, there is a sidebar with icons for a menu, search, a variable '[x]', a key, a folder, and a double arrow. The main content area shows a section titled 'Autoencoders' with a downward arrow. Below the title, there is a paragraph of text: 'This code includes implementations of various simple autoencoders using the Keras front end to TensorFlow. We'll run all of the code on the MNIST data to gain a basic understanding of what everything is doing. You'll then run this on the Fashion MNIST data.' Below the paragraph, there is another paragraph: 'The starter code for this demo is from <https://github.com/ardendertat/Applied-Deep-Learning-with-Keras>. For Keras documentation, see <https://keras.io>.'

Recap from last class

Language models

Modern language models

The algorithm assigns a “score” to possible next words w :

$$\text{score}(\underset{\substack{\uparrow \\ \text{possible next word}}}{w}; \overbrace{w_1, \dots, w_n}^{\text{previous words}})$$

- score is computed as the similarity between two high-dimensional vectors—we’ll explain how!
- it is then converted into a probability

Key steps

- *Tokenize* the text into a fixed vocabulary
- *Embed* the tokens into high-dimensional vectors
- *Mix* the embeddings in the context using “Attention”
- *Map* the resulting vectors using a neural network

The vocabulary problem

- Word embeddings we saw constructed for 400,000 word vocabulary
- We each know $\approx 40\text{K}$ words
- What if a word is not in the vocabulary?
- Need to be able to process—and generate—arbitrary text

Tokenization

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

Billy was a very baaaaaaaaad goat.

This is another \$%^*ish example.

This is what we see

Tokenization

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

Billy was a very baaaaaaaad goat.

This is another \$%^*ish example.

Text

Token IDs

This is how the tokenizer groups bytes into tokens

Tokenization

```
[77845, 19209, 84, 12, 5519, 11, 264, 4647, 14946, 3637, 304, 279, 15013, 27286, 12818, 315, 9853, 12167, 11, 706, 25281, 369, 220, 7994, 1667, 11, 449, 264, 29869, 5536, 389, 1202, 4029, 382, 97003, 574, 264, 1633, 293, 29558, 33746, 329, 54392, 382, 2028, 374, 2500, 400, 46999, 9, 819, 3187, 382]
```

Text

Token IDs

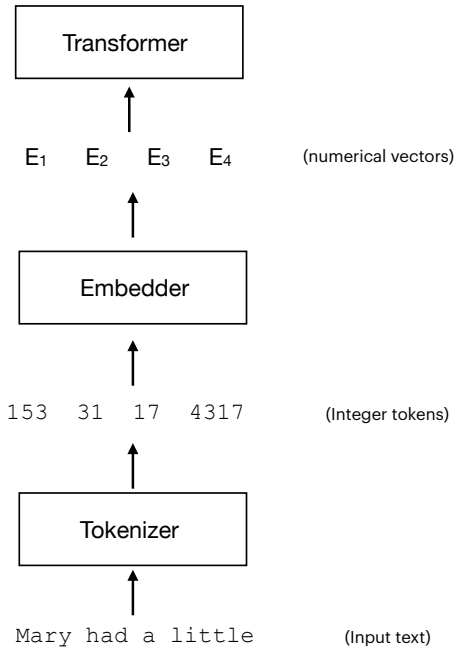
This is what the LLM sees—the “atoms” of an LLM

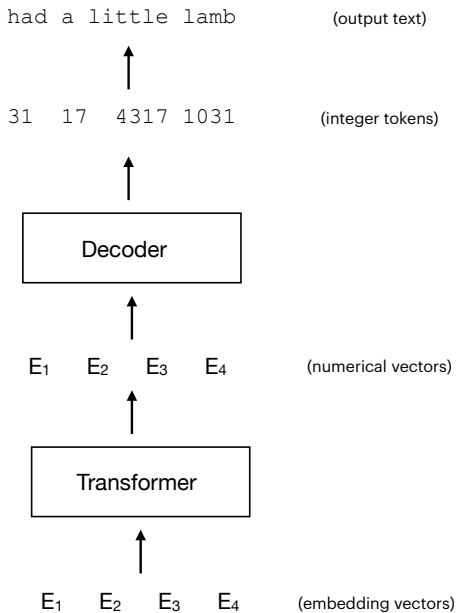
Tokenizer properties

- *Reversible*: Can convert from tokens back to original text
- *General*: Works on arbitrary text, even very different from training
- *Compressive*: Each token is about 4 bytes
- *Statistical*: Will break strings into frequently occurring pieces

BPE Algorithm – Byte Pair Encoding

- ① Input: Large sequence of bytes (UTF-8 encoding)
- ② Initialize: Each byte is a token; $V=256$
- ③ Iterate for K steps:
 - i Find pair of tokens (s, t) with largest count of co-occurring
 - ii Replace each occurrence of (s, t) with V
 - iii $V \leftarrow V + 1$
- ④ Final vocabulary size: $256 + K$





Key problem

Long distance relationships

The key problem is integrating information across long distances

The leaves, as the weather begins to warm up New Haven, silently on the in .

The key problem is integrating information across long distances

The leaves, as the weather begins to warm up in New Haven, grow silently on the trees in April.

The key problem is integrating information across long distances

The leaves, as the weather begins to warm up in New Haven, grow silently on the trees in April.

To predict `grow` and `trees`, the model needs to refer back to `leaves` in memory (context):

- ▶ It's the subject of the sentence, and plural — syntax
- ▶ It has a “foliage” meaning — semantics

The leaves, as the weather begins to warm up in New Haven,

The leaves, as the weather begins to warm up in New Haven,

Intuition:

Embedding vector $\varphi(\text{leaves})$ is added to the vector used to fill in the first blank.

Since $\varphi(\text{leaves})$ is similar to $\varphi(\text{grow})$, this boosts up the chances of generating `grow`

The leaves, as the weather begins to warm up in New Haven, grow silently on the

The leaves, as the weather begins to warm up in New Haven, grow silently on the

Intuition:

Embedding vectors $\varphi(\text{leaves})$ and $\varphi(\text{grow})$ are then added to the vector used to fill in the next blank.

This boosts up the chances of generating `trees`

Attention

- Which embeddings are added to current position is determined by *Attention*
- Attention is essentially a (very large) system of linear equations
- The weights in the linear system are optimized to extract the most predictive properties

Attention

- Which embeddings are added to current position is determined by *Attention*
- Attention is essentially a (very large) system of linear equations
- The weights in the linear system are optimized to extract the most predictive properties
- Attention is organized into multiple “heads”
 - ▶ Some may capture “syntax” (`subject` $\rightarrow$ `verb`)
 - ▶ Some may capture “semantics” (`leaves` $\rightarrow$ `trees`)
 - ▶ But they are generally very difficult to interpret!

Attending to specific things



Such relations can be extracted with a linear transformation (matrix multiply)

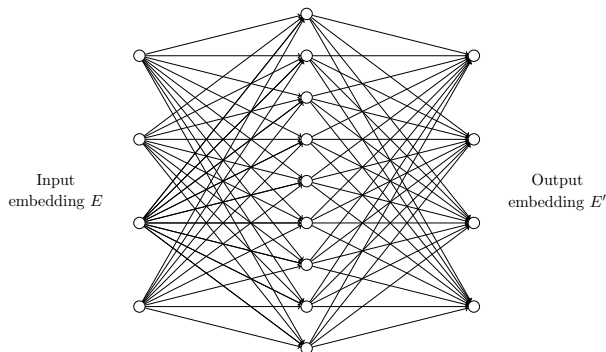
More intuition



- Different attention vectors are concatenated
- Capture different meanings of the sequence
- Words “hungry” and “sweet” are predicted using all of them

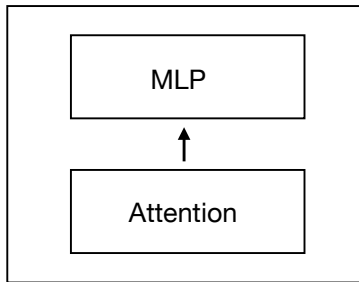
Adding nonlinearities

The last step is to take the resulting vectors (after they have been mixed up), and map them to new vectors using an MLP:

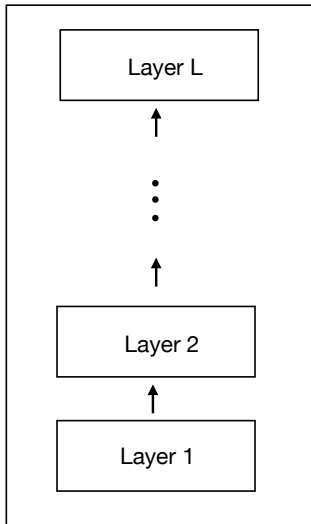


Creates new “features” to be used as embeddings in the next layer

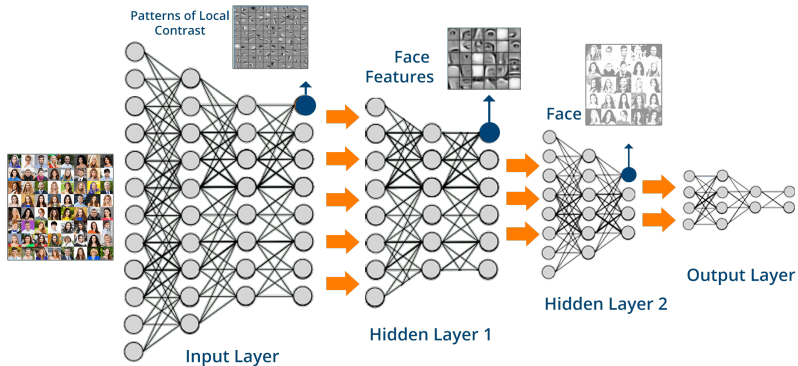
Transformer Layer



Transformer

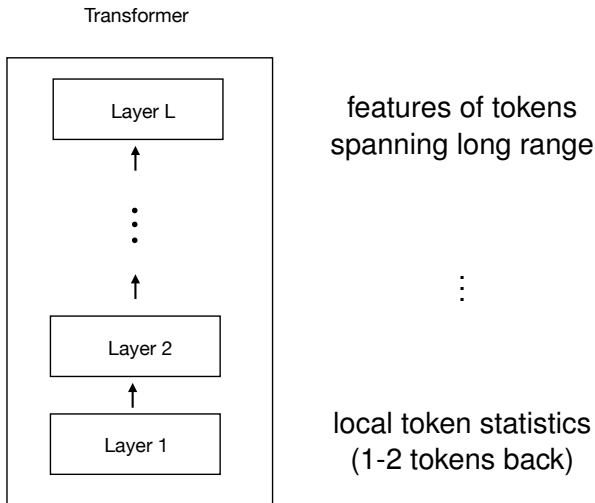


Increasing specificity of patterns



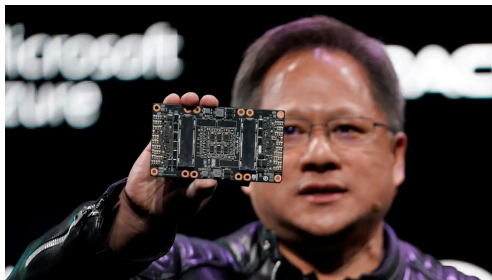
- Early layers capture local patterns of image contrast
- Later layers capture larger patterns that match parts of faces

Increasing specificity of patterns



Computation

- Everything is implemented as big matrix multiplies
- Carried out very efficiently on GPUs
- GPUs are optimized for linear equations (vector graphics)
- Allows scaling models to very large size



The residual stream

For *each* occurrence of a token in context, the GPT-3 Transformer maps it to 97 *different* embeddings (aka encodings)

- The first embedding represents the general token, across all contexts — 12,288 trainable numbers
- Each of the 96 layers then transforms this embedding using Attention and an MLP
- Result is $96 \times 12,288$ numbers for each token occurrence — more than one million total numbers (2 million bytes)
- Not *directly* interpretable by humans; but can be probed

The residual stream

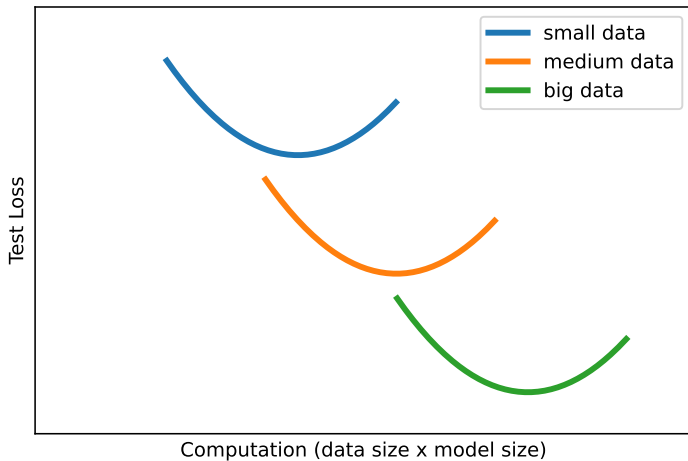
The context length for GPT-3 is 2,048 tokens

- Total numbers for each context: $2,048 \times 1.1 \text{ million} \approx 2 \text{ billion}$
- The numbers for previous layer are added to the numbers for the next layer
- This is called the *residual stream*
- Number of parameters in model: 175 billion
 - ▶ We'll describe how to count parameters next

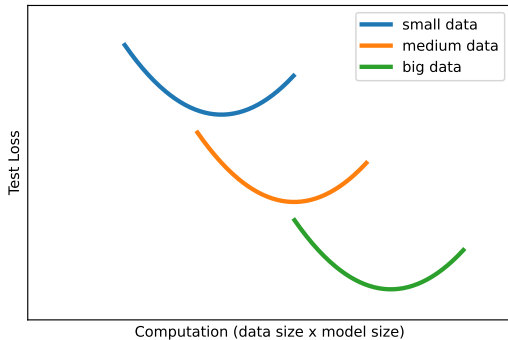
The Transformer architecture

How large are the models?

Scaling behavior of ML models



Scaling behavior of ML models



How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Embeddings have $d \cdot V$ parameters
 - ▶ d numbers for each token, V tokens

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Self-Attention has $\approx 4d^2$ parameters
 - ▶ $d \times d$ matrix for queries (across heads)
 - ▶ $d \times d$ matrix for keys (across heads)
 - ▶ $d \times d$ matrix for values (across heads)
 - ▶ $d \times d$ matrix for combining heads

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- MLP applied after has $\approx 8d^2$ parameters
 - ▶ Two layers
 - ▶ Hidden layer has $4 \cdot d$ neurons
 - ▶ Output layer has d neurons
 - ▶ $4d^2$ weights in each layer; $8d^2$ weights overall

How does size of LLM scale?

Design choices: number of tokens V , dimension d of embeddings, and number of Transformer layers L

- Embeddings have $d \cdot V$ parameters
- In each Transformer layer:
 - ▶ Attention has about $4d^2$ parameters
 - ▶ MLP has about $8d^2$ parameters
- Total trainable parameters: $12d^2 \cdot L + d \cdot V$

Size of models

GPT-2 (2019)

$$d = 1,600$$

$$V = 50,257$$

$$L = 48$$

$$12 \cdot d^2 \cdot L + d \cdot V \approx 1.5 \text{ billion parameters}$$

- ▶ Each GPT-2 Transformer layer has about 30 million parameters

Size of models

GPT-3 training data^{[1]:9}

Dataset	# tokens	Proportion within training
Common Crawl	410 billion	60%
WebText2	19 billion	22%
Books1	12 billion	8%
Books2	55 billion	8%
Wikipedia	3 billion	3%

Size of models

GPT-3 (2020)

$$d = 12,288$$

$$V = 50,257$$

$$L = 96$$

$$12 \cdot d^2 \cdot L + d \cdot V \approx 173 \text{ billion parameters}$$

- ▶ Each GPT-3 Transformer layer has about 1.8 billion parameters



Original painting by Johannes Vermeer; transformed (pixelated) by acagastyu, CC0, via Wikimedia Commons

ANNALS OF ARTIFICIAL INTELLIGENCE

CHATGPT IS A BLURRY JPEG OF THE WEB

OpenAI's chatbot offers paraphrases, whereas Google offers quotes. Which do we prefer?

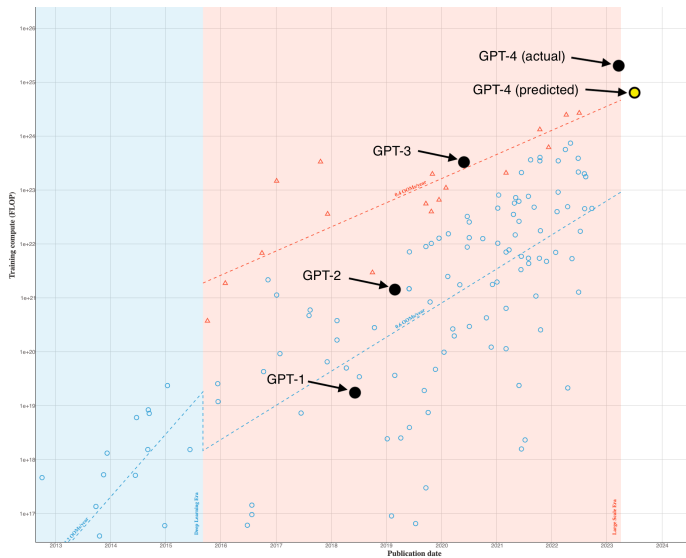
By Ted Chiang

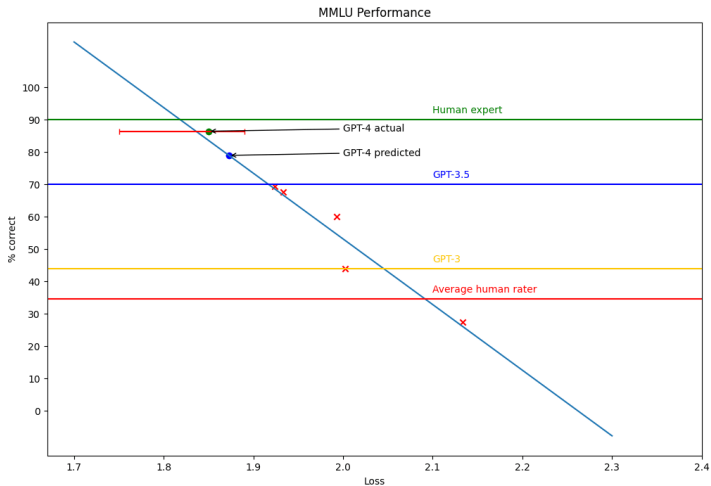
February 9, 2023

Back of the envelope calculation



- LLM loss on test data is ≈ 1.9 nats/token
- Translates to entropy of about $1.9 / \log_e(2) \approx 2.74$ bits/token
- So, *lossless* coding of 500 billion tokens takes ≈ 1.4 trillion bits
- GPT-3's 175 billion params ≈ 1.4 trillion bits (8-bit precision)
- GPT-3 could, in principle, memorize most of the internet!





MMLU: Massive Multitask Language Understanding, <https://en.wikipedia.org/wiki/MMLU>,
<https://paperswithcode.com/dataset/mmlu>

Sutton's "Bitter Lesson" (2019)



“The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”